Quantum Machine Intelligence (2021) 3:10
https://doi.org/10.1007/s42484-021-00039-9

RESEARCH ARTICLE

Optimizing the optimizer: decomposition techniques for quantum annealing

Gideon Bass[1] · Maxwell Henderson[2] · Joshua Heath[1] · Joseph Dulny III[1]

Received: 27 July 2020 / Accepted: 25 January 2021
© The Author(s), under exclusive licence to Springer Nature Switzerland AG part of Springer Nature 2021

Abstract
Although quantum computing hardware has evolved significantly in recent years, spurred by increasing industrial and government interest, the size limitation of current generation quantum computers remains an obstacle when applying these devices to relevant, real-world problems. In order to effectively exploit the potential benefits of quantum computing, heterogeneous approaches that combine both classical and quantum computing techniques are needed. In this work, we explore multiple heterogeneous algorithms for solving a variety of industry-relevant benchmark problems in order to understand how best to leverage quantum computers given current constraints. Our results indicate the following: (1) that solver performance is highly dependent on the structure (size and edge density) of the problem graph; (2) that reusing a single fixed problem embedding, as opposed to dynamically searching for problem embeddings, is key to avoiding computational bottlenecks; (3) that solutions of better quality are produced by algorithms that iteratively propagate the influence that solving an individual sub-problem has to the remainder of the larger problem; and (4) that the Qbsolv algorithm (which implements the aforementioned techniques) is, at this time, the state-of-the-art in producing quality solutions, in a timely fashion, to a variety of theoretical and real-world problems too large to directly embed onto a quantum annealing device.

Keywords Optimization · Quantum · Quantum computing · Quantum annealing

1 Introduction

The field of quantum computing is entering the so-called NISQ-era of Noisy, Intermediate-Scale Quantum computers (Preskill 2018). While the NISQ-era is expected to yield quantum computers that can surpass certain computational benchmarks, their applicability to real-world problems will still be constrained by current generation hardware limitations. Currently, the largest (in terms of qubit count) commercially available quantum computing hardware device is the D-Wave 2000Q quantum annealer (D-Wave 2018), which has 2048 qubits when functioning at full capacity and is designed to solve a particular class of optimization problems (Farhi et al. 2001; Garey and Johnson 1990) using a quantum annealing algorithm (Farhi et al. 2001; Brooke et al. 1999; Finnila et al. 1994; Kadowaki and Nishimori 1998; Santoro et al. 2002; Santoro and Tosatti 2006). While this device showcases the rapid hardware advances made in the field from the first 128-qubit commercially available device offered in 2011, current hardware is vastly exceeded by the thousands or millions of variables that real-world problems often require.

A common workaround to this limitation has been to investigate the scalability of quantum applications using small exemplar problems. Performance results generated from these investigations are then used to extrapolate algorithm performance on larger devices. While scalability studies are vital for understanding longer-term quantum computing viability, they do not answer the question: "What is the optimal way to use currently available quantum devices to solve real problems today?"

A promising path forward for mitigating quantum hardware restrictions is to explore heterogeneous classical-quantum computing solutions. Heterogeneous approaches harness the strengths of both classical and quantum systems for solving difficult problems by utilizing the quantum component as a specialized sub-processor within a larger classical algorithm. Several previous projects have evaluated heterogeneous approaches using existing quantum

✉ Gideon Bass
bass_gideon@bah.com

[1] Booz Allen Hamilton, Washington, DC, USA

[2] Rigetti Computing, Berkeley, CA, USA

Published online: 26 March 2021

annealing devices (Bass et al. 2018; Bian et al. 2014, 2016; Chapuis et al. 2019; Djidjev et al. 2016, 2018). However, many of these algorithms were developed for specific problems—a paradigm that is exemplified in Ajagekar et al. (2020). That work, while a thorough examination of problem decomposition techniques across multiple problems, focused on developing targeted decomposition algorithms, each of which was aimed at and evaluated on one of the particular problems examined. An exhaustive analysis of the performance of various heterogeneous techniques—both existing and new—when solving a variety of diverse problems is currently absent from the literature.

The goal of this work is to provide a more comprehensive understanding of best practices for heterogeneous quantum annealing algorithms to solve industrially relevant optimization problems with different problem graph characteristics. We accomplish this through a multivariate evaluation of four heterogeneous methods on four problems, each of which is relevant to a real-world application and has a unique underlying graph structure. As discussed above, most earlier work in this field has focused on decomposition algorithms for specific problems. This work is significant in its progress toward developing and comparing generalized heterogeneous algorithms capable of solving any arbitrary quadratic unconstrained binary optimization (QUBO) problem—the native problem format of current quantum annealing devices.

Again, we would like to emphasize that the goal of this work is not to compare quantum annealing to state-of-the-art classical optimization. Instead, our work is focused on comparing different ways of using existing quantum annealing hardware, and comparing them to each other, to drive closer to understanding the best way to use the hardware for optimization problems. Only once that is known will it be possible to do a fair comparison with classical techniques.

As the first true "quantum benefit" will likely result from a highly specialized application (e.g., Arute and et al 2019), determining how to optimize for every level of the quantum computing stack is pivotal. While heterogeneous solutions will likely not provide exponential quantum speedups in the near term, they have the potential to offer sub-exponential improvements in algorithm performance for valuable real-world problems. This is not negligible, as even minor algorithmic improvements as a result of leveraging quantum could translate to notable industrial or military benefit, subsequently spurring resource devotion to quantum computing research. We believe that there will always be a virtuous cycle between scientific progress and industrial/defense applications, and finding real-world applications for quantum annealing will result in significant scientific advances.

In addition to optimization problems, efficient approximations of these native QUBOs have shown multiple applications to machine learning use cases, such as for learning kernel (Noori et al. 2020), boosting algorithms (Neven et al. 2012), and sampling (Henderson et al. 2019).

While this study's focus is primarily on evaluating heterogenous algorithms that leverage quantum annealing, further studies better characterizing the performance using other forms of optimization should be investigated. A recent example highlighting this importance is the work of Job and Adachi (2020); this research showed that a previous algorithm for training deep belief networks hinting at a potential benefit using a quantum annealing over Gibb's sampling techniques dissipated when compared against the same algorithm using simulated annealing.

This work is organized as follows: In Section 2, we discuss the overall experimental setup, including the specifics of the problems and heterogeneous techniques used. In Section 3, we present and discuss the results of the many experimental runs we performed. Finally, Section 4 summarizes our conclusions and provides overall commentary on the current state-of-the-art in heterogeneous optimization, as well as ideas for improvements to current and future techniques.

2 Methods

2.1 Quantum heterogeneous optimization techniques (QHOTs) framework overview and abstract class descriptions

To accomplish our goals of exploring and fairly comparing the performance of different heterogeneous optimization techniques when solving different types of problems, we developed an extendable, object-oriented software framework, whose design is inspired by the following problem-solving workflow:

1. A type of *ProblemGenerator* is initialized;
2. A specific *QuboProblem* instance is generated using the generator; and
3. The problem instance is submitted to and solved by a *HeterogeneousSolver*.

These three components form the backbone of the QHOTs framework and are each implemented in a separate abstract class which establishes concrete class implementation requirements and collectively define the integration points between software components. The framework is easily extendable; new types of problems and heterogeneous techniques need only satisfy integration point and class requirements.

Each optimization problem solved using the QHOTs framework is first mapped to a QUBO problem representation. By definition, a QUBO problem has the following formulation:

$$E(S) = \sum_i a_i q_i + \sum_{i<j} b_{i,j} q_i q_j,$$

where the function E evaluates a solution vector S: a set of binary variables $S = \{q_1, q_2, ..., q_N\}$, wherein each q_i represents the measured value of a logical qubit. Mapping an optimization problem into the QUBO format involves calculating the appropriate values of the a_i's (known as the *qubit biases*) and $b_{i,j}$'s (known as the *coupling strengths*) of E such that a solution S that minimizes E will represent an optimal solution to the original optimization problem formulation. In this work, a QUBO *problem graph* is defined by the graph in which nodes represent logical qubits and an edge is present between any pair of logical qubits that share a non-zero coupling strength.

Once a ProblemGenerator has instantiated a QuboProblem, a solution is determined by submitting the problem to a HeterogeneousSolver. The HeterogeneousSolver uses a mix of classical techniques and quantum annealing to solve the overall optimization problem, with the one commonality being that the approach prepares a set of sub-QUBOs (QUBO problems that are reduced versions of the full problem) which are embedded on and solved directly by a quantum annealing device.

2.2 Optimization problems

In this section, we describe key aspects of each problem type evaluated in this work.

While the problems in this study were generally in the class of NP-hard problem space (traveling salesman, job shop, etc.), there is an interesting research question in verifying the computational complexity of any particular instance or sub-problem after decomposition by the various HOTS. This investigation was outside the scope of this work but would provide another strong foundation for ensuring that a fair comparison is made between HOTS in the future.

2.2.1 Random QUBO with time-domain structure

Many optimization problems have inherent time-based constraints. When mapping such time-constrained problems to the QUBO form, the presence of these time constraints results in the underlying problem graph having a structure similar to that of some deep neural architectures—specifically, the Deep Boltzmann Machine (Salakhutdinov and Hinton 2009). Because of this similarity, problems of this type in this work are referred to as Deep Boltzmann Graph (DBG) problems.

In these problems, we simulate time-domain structure by grouping variables into layers, representing different moments in time, and forming constraints (represented by non-zero coupling strengths) between variables across some finite number of layers (time steps). For example, take a simple instance of the well-known Job Shop Scheduling problem, in which we use i machines to complete j jobs at various t times. When considering the corresponding QUBO problem graph, a natural structure arises wherein each time t is represented by a corresponding layer of variable nodes, and intra-layer constraints (e.g., to enforce that the same job/machine is not being completed/used at the same time) and inter-layer constraints (e.g., to enforce that a machine performing the same job across multiple time steps is not simultaneously assigned another job) form.

With a strong time-domain constraint, variables are restricted to only having non-zero coupling strengths with variables within adjacent layers. Loosening this constraint to allow non-zero coupling strengths between variables two or more layers apart allows the emphasis of time to be steadily lowered. At the extreme, when any two variables can be connected, random unstructured QUBOs with up to fully connected problem graphs can be generated.

2.2.2 Symmetric travelling salesman problem

One of the most well-known combinatorial optimization problems is the Travelling Salesman Problem (TSP). Given an undirected graph with weighted edges, an optimal solution to the Symmetric TSP is a path through all graph vertices such that each node is visited exactly once and the sum of the weights of edges traversed in the path is minimal. In this work, we implement the QUBO formulation for the Symmetric TSP given in Lucas (2014).

We explore the TSP on two types of graphs. The first, "Random," is fully connected graphs defined on 11, 20, 50, or 100 vertices, with edge weights being randomly sampled from the range [1,10]. The second type of graph is also fully connected, with each graph being taken from a unique benchmark TSP instance downloaded from TSPLIB. Reinelt (1995).

2.2.3 Satellite sub-constellation assignment

The Satellite sub-Constellation Assignment (SCA) problem involves partitioning a set, referred to as a "constellation," of N satellites into k smaller "sub-constellations." Each SCA problem instance is defined by N, k, and the "coverage" of each possible sub-constellation, defined as the percentage of time that an Earth region of interest is within signal range of at least one satellite within that sub-constellation. The goal is then to assign each satellite to exactly one sub-constellation such that the total sum of the coverages of each

of the sub-constellations is maximized. This problem can be cast as a weighted k-clique problem, which can then be reformulated as a QUBO.

In weighted k-clique problem formulation, each possible sub-constellation is represented by a single node in a graph, with a node weight equal to the coverage of the sub-constellation that it represents. An edge is present between two nodes if the two sets of satellites are disjoint, i.e., there is no satellite that is in both sub-constellations. Within the resulting graph, a clique of size k represents k disjoint sub-constellations. The goal is to find the clique of size k that maximizes the sum of node weights. For a description of the process by which the weighted k-clique problem can be formulated as a QUBO, see Bass et al. (2018).

As with all types of problems explored within this work, the size of the problem is a serious consideration. This is especially true of the real-world SCA problem, which contains over one million nodes when formulated as a weighted k-clique problem. Because the QUBO representation of the weighted k-clique problem requires a fully connected problem graph (i.e., a non-zero coupling strength exists for every pair of qubits), the problem requires over one million logical qubits and over 10^{14} unique coupling strengths. Simply generating the problem coefficients required traditional memory requirements greater than the classical computing resources allocated for this work. To mitigate these memory issues, it was observed (Bass et al. 2018) that solutions of comparable quality could be found by ignoring sub-constellations with coverages below a given threshold. Empirically, we found that devising a threshold that removed all but the 99th percentile of sub-constellation coverages shrunk the problem to a more manageable size. Additionally, for the 27-satellite problem instance explored in this work, we found that limiting sub-constellation size to 3, 4, or 5 and setting k to 7 produced solutions of good quality. While still too large to directly embed on a quantum annealer, the reduced problem is small enough to be easily manipulated in classical memory by our QHOTs.

2.2.4 Military maintenance planning and scheduling

Another example of a real-world optimization problem that we explore in this work is the Maintenance Workload Problem (MWP) introduced in Squires and Hoffman (2014). This problem is a specific instance of the more general job shop scheduling problem, which has applications in many areas of industry and defense. Given a set of items that require repairs, the MWP asks for the most cost-effective scheduling of these items at repair facilities that satisfies the following constraints:

1. Each item originates from a specified location during a specific week in time (release date) and must arrive, repaired, at a specified location before a specific week (due date).
2. Each item must be shipped from its origin to the selected repair facility, and from this repair facility to its final destination, with the financial and temporal costs of these shipments being considered.
3. During a given week, each repair facility has a limited number of worker-hours that can be used to repair items.
4. Each repair is of a certain type and must be completed (without preemption) at a facility that can perform that type of repair, where the length (in number of weeks), cost, and required worker-hours per week for the repair are unique to the facility.

In order to cast the MWP as a QUBO problem, we represent a potential solution (scheduling) by the combination of values taken by binary variables $x_{(R_i,L_j,T_k)}$, where $R = \{R_1, ..., R_X\}$ is the set of all repairs, $L = \{L_1, ..., L_Y\}$ is the set of all repair facilities, and $T = \{T_1, ..., T_Z\}$ is the set of all weeks between the earliest release date and the latest due date of the repairs in R. In this definition, $x_{(R_i,L_j,T_k)} = 1$ if repair i is going to be completed at repair facility j and will begin to be completed (i.e., shipped to j) on week k, and $x_{(R_i,L_j,T_k)} = 0$ otherwise.

The mathematical formulation of each of the aforementioned constraints is found in Squires and Hoffman (2014). To map this formulation to a QUBO, we first modified each constraint such that a solution that satisfies a given constraint would minimize the QUBO definition of that constraint. These individual constraints were added together to form the complete QUBO formulation of the MWP. This QUBO requires on the order of $R \cdot L \cdot T$ logical qubits, though the actual number may be less because $x_{(R_i,L_j,T_k)}$ will always be 0 for combinations of job/facility/release date that are impossible.

2.3 Quantum heterogeneous optimization techniques

In this section, we detail key algorithmic steps that define each heterogeneous technique investigated within this work. Each algorithm implemented is reusable in that it can be used to solve any QUBO problem, including each of the problems described in 2.2.

2.3.1 Principal component decomposition (PCD)

Although the PCD algorithm can calculate global solutions for QUBOs of arbitrary size and structure, it was developed particularly for application to problem graphs with time structure (e.g., Job Shop Scheduling problem). The PCD algorithm makes a series of cuts along the temporal axis

of the problem graph, where each cut removes edges that intersect the cut. This process is repeated until each of the resulting disconnected graph components, representing subproblems of the original QUBO, is directly embeddable on an available quantum annealing device. This approach maximally preserves the initial temporal structure of the QUBO into each subproblem by attempting to find an appropriate hardware embedding after each additional graph cut. As this study endeavors to compare the different QHOTs in a black-box manner, we assume only the coefficients of the QUBO formulation are provided. To determine where to cut the problem graph without access to any explicit structural information a priori, the PCD algorithm takes two primary steps:

1. *Spring Layout algorithm.* To organize the graph, the spring layout algorithm of NetworkX (Hagberg et al. 2008), based off the force-directed Fruchterman-Reingold Algorithm (Fruchterman and Reingold 1991), is used to generate an initial 2D-coordinate pair for each graph node. The algorithm determines the 2D-coordinates through a simulation process, wherein the graph nodes undergo Brownian motion in an effort to diffuse as much as possible. Nodes connected by an edge are bound by a constraining force causing the overall graph layout to converge, after many simulation steps, such that nodes sharing many mutual connections gravitate toward each other. As stated in Fruchterman and Reingold (1991), the exact convergence criterion for the number of outer loop iterations is a complex question, but heuristically we have seen that 1000 simulation iterations provide efficient results.
2. *Principal Component Analysis (PCA).* After the initial graph layout has been calculated using the spring layout algorithm, we apply PCA to the 2D coordinates of all the nodes in the graph, to ensure that we are slicing the graph along the primary axis. We then use these components as the final graph coordinates; see Fig. 1 for examples of final graph coordinates generated from 4 different input graphs. After this transformation, the problem graph is partitioned, moving from left to right along the graph coordinates to determine subgraphs. If a subgraph cannot be embedded due to size or connectivity issues, it is continually sliced along the primary axis until an embedding of each of its components is found. These sub-QUBOs, defined by these embeddable subgraphs, are then sent to the quantum annealer.

After generating solutions for all subproblems using the quantum annealing device, the PCD algorithm applies a greedy recomposition algorithm to combine all subproblem solutions into a global solution. Specifically, all subproblem solutions are concatenated to form a single solution. Then, we iterate through all QUBO problem constraints (i.e., non-zero coupling strength between qubits) to ensure they are satisfied. For qubit pairs that break a constraint (i.e., both qubits are measured as 1), we randomly change one of the values to 0 to satisfy the constraint.

2.3.2 Freeze-and-Anneal (FA)

FA is a novel heterogeneous optimization technique that extends the authors' prior work (Bass et al. 2018). The previous iteration of this algorithm was designed specifically to solve the SCA problem described in Section 2.2.3, while the FA algorithm presented in this work has been generalized and is capable of generating solutions for arbitrary QUBO problems.

FA begins by using a genetic algorithm (GA) (Goldberg and Holland 1988; Holland 1992) to evolve a population of potential solutions to the QUBO problem. The size of this population and the number of generations for which the GA will evolve are parameters explored in this work. Specifically, we experiment with the former set to 250 and 500, and the latter set to 10, 50, and 100. The "fitness" of each solution is dependent on the corresponding energy of the QUBO for that solution; lower energy implies higher fitness. Because a solution is simply a collection of values of the QUBO's binary variables, each can easily be represented as a bit string (e.g., 00101001). After the population is allowed to evolve for a pre-specified number of generations, we iteratively perform the "freeze" step to identify consistent variables. In this step, we analyze the population of all of the final bitstrings, looking for which variable is the most well-predicted. This per-variable frequency of a value (0 or 1) is weighted by the population-wide frequency of the respective value.

Each time a variable is frozen, the QUBO problem is modified by removing that variable. Any variables that couple with the frozen variable have their qubit biases increased by the respective coupling strength if the variable is frozen to "1," or left unchanged if the variable is frozen to "0." The freezing process steadily reduces the size of the remaining QUBO until it is small enough to be directly embeddable on the available quantum annealing device. In designing this algorithm as such, we hypothesize that this reduced QUBO consists of the binary variables that are the "most difficult" for the classical technique to evaluate, and is therefore assumed to be the "hardest part" of the larger problem. The rationale is that a quantum advantage can best be achieved by focusing on this hardest part. The solution returned by the quantum annealer is then concatenated with the values of the frozen variables to return the final combined solution.

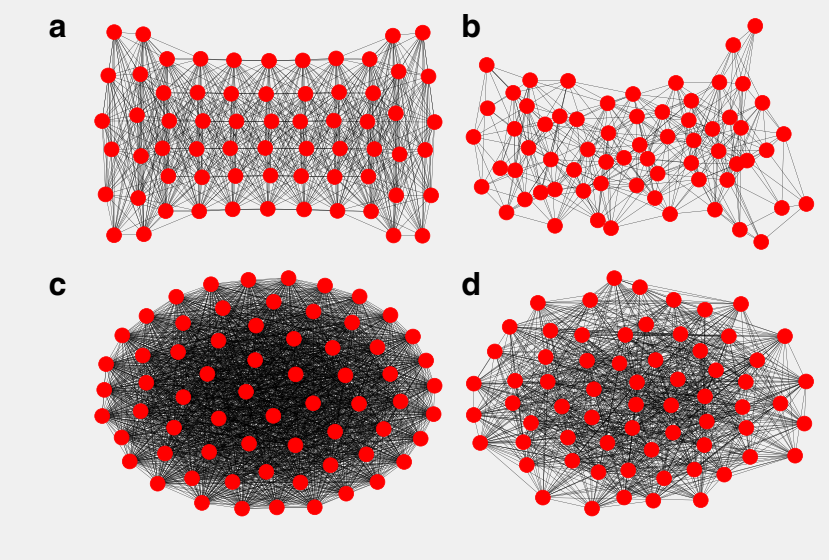


Fig. 1 Final node positions determined using the PCD algorithm for 4 example DBG problem graphs defined on 11 layers of 6 nodes each. In graphs A and B, constraints can occur between nodes that are within two layers of each other; this restriction is non-existent in graphs C and D. For those constraints that can occur in these graphs, there is a 0.5 probability that a constraint between two variables does occur in B and D, and a probability of 1 for A and C. As seen in A and B, the PCD algorithm is able to regenerate the original time-domain structural patterns of the input problem graphs, and struggles to find any such structural patterns in the random graphs of C and D

2.3.3 Qbsolv

The open-source Qbsolv algorithm, outlined in Booth et al. (2017), was initially created for the purpose of solving QUBO problems too large to natively fit on quantum annealing hardware. The Qbsolv algorithm leverages a quantum annealer's ability to heuristically search large solution spaces, while also accounting for the annealer's lack of precision through the use of a more precise, local minimization algorithm (namely, Tabu search). In this work, analysis of Qbsolv was done so using Version 0.2.10 of the algorithm. We explore a single Qbsolv parameter, $NumRepeats$—the number of main loop iterations that need to pass with no solution improvement before the algorithm is terminated, which we assign values of 1, 3, 10, 30, and 50.

2.3.4 Iterative Centrality Halo (ICH)

The Iterative Centrality Halo algorithm implemented in this work is an adaptation of the previous Core-Halo (CH) work of Chapuis et al. (2019), in which CH partitioning (originally introduced in Djidjev et al. (2016)) is applied to decomposing the problem graph of a QUBO representation of the Maximum Clique problem. The primary algorithmic steps of the ICH algorithm are as follows:

1. Define max_nodes as the size of the largest fully connected problem graph that can be embedded onto your quantum annealer (on the 2000Q, this is 65).
2. Given a QUBO Q with problem graph G, let S be a global solution vector with values of all variables initialized to 0.
3. Let C be the node of highest degree in G, which we define as the "central node."
4. Let H be the set of nodes connected to C. If $|H| > max_nodes - 1$, randomly remove nodes from H until $|H| = max_nodes - 1$.
5. Let Q_{sub} be the sub-QUBO defined on C and the nodes within H. Find an embedding of Q_{sub} onto the quantum annealing hardware and solve.
6. Set the values of Q_{sub}'s corresponding variables in S to the values returned by the annealer.
7. Propagate, through Q, the influence of setting these variables by modifying the qubit bias of each variable connected to a variable set to 1 by the coupling strength between these variables.
8. Remove C and the nodes within H from G.
9. Repeat steps (iii)—(viii) until the entire problem graph has been consumed.

ICH departs significantly from the original algorithm in several ways, predominately in (1) the use of centrality to select central nodes which prioritizes nodes that likely have a more significant impact on the global solution, and (2) the iterative subgraph solution clamping process, which transfers information about the current global solution into the remaining QUBO.

2.3.5 Random with greedy classical post-processing (RGCP)

The intent of this work is not to form direct comparisons of QHOTs against state-of-the-art classical solvers, but rather to understand the benefits and detriments of using various QHOTs to solve particular problems. However, it remains necessary to establish a baseline comparison of classical performance. In terms of solution quality and constraint violation, the results returned from the RGCP solver act as this baseline. When given a QUBO problem to solve, the RGCP solver simply generates a random solution (i.e., all problem variables are assigned a random binary value) and leverages greedy classical optimization to return the nearest feasible solution. We perform this classical post-processing (further detailed in the following section) on solutions returned from each of the other solvers as well to ensure a set of comparable, feasible solutions. The "fixed" solution returned from the RGCP solver serves to normalize the impact that this post-processing has on the quality of the other solvers' fixed solutions.

3 Results

3.1 Experimental setup

The goal of this effort was to exhaustively compare the performance of various QHOT solvers on various types of optimization problems. In total, 104 unique problem instances were generated and solved (88 DBG, 14 TSP, 1 SCA, and 1 MWP). Problem graph characteristics, such as graph size (number of logical qubits) and edge density (ratio of the number of edges that do exist to the number of edges that could exist), are recorded for each problem. It should be noted that no attempt is made, in this work, to identify the true optimal solution to any problem explored; therefore, the quality of a solution found using a certain QHOT should only be evaluated relative to the quality of solutions found by other QHOTs. Twelve unique QHOT instances were used to solve each problem (1 PCD, 4 FA, 5 Qbsolv, 1 ICH, and 1 RGCP). Note that multiple versions of the FA and Qbsolv algorithms are tested because we chose to explore the parameter space of these algorithms. Through the remainder of this section, we refer to an "experiment" as a combination of (1) the type of problem being solved; (2) the parameter values that were used to generate the problem instance; (3) the QHOT being used to solve the problem; and (4) the parameter values of the QHOT. A complete list of all experiments conducted in this effort, as well as various performance metrics captured during the execution of each, are provided in the supplemental data.

All experiments used the D-Wave 2000Q (maintained by NASA/USRA) as a quantum sub-processor. Experiments in which the DBG, TSP, and MWP problems were solved leveraged an Amazon Web Services (AWS) instance (c5d.xlarge) for classical processing. Due to data restrictions, experiments in which the SCA problem was solved leveraged the following classical processor: Intel Core i5-5300 at 2.3 GHz. As such, time-based metrics for the SCA experiments are not comparable to that of other problems explored in this work.

3.2 Evaluation criteria

To assess QHOT performance, the following metrics were primarily used:

1. *Time-to-solution*. Understanding the total time needed to find a solution to a problem can be important for practical applications, and is essential for those that require near "real-time" results. An experiment's *time-to-solution* is the amount of time (in seconds) that the QHOT expended before returning a solution to the QUBO problem being solved. While any method leveraging cloud-based quantum computing will execute at far from "real-time" speeds, we chose a maximum time threshold to simulate some arbitrary time constraint that an end user may have for their application. If an experiment eclipsed this threshold, which naturally happens as problem sizes scale up, the experiment would be halted. In the interest of time, our experimental threshold was set at 30 min, which allowed the QHOTs to generate solutions for most experiments while avoiding extremely long runtimes.
2. *Relative run-time*. In addition to observing which experiments were able to complete within this threshold, it is also interesting to record the proportional time spent on *classical*, *embedding*, and *quantum* tasks. These refer (respectively) to:
   (a) Computational time spent on the classical outer-loop parts of a given QHOT;
   (b) Computational time spent using the heuristic problem embedding finding tool, minorminer (Cai et al. 2014), to map sub-QUBO problems onto the quantum annealing device; and
   (c) The total amount of time spent waiting on the quantum annealer—from submission of a sub-QUBO (and embedding) to the device until a solution response is returned (including time associated with network latency, the device's queue, and the annealing process). The amount of time per request is generally uniform across all conducted

experiments; therefore, an experiment's *quantum* time is directly proportional to the number of requests being made to the quantum annealer within that experiment.

As the different QHOTs employ unique classical algorithms, handle embeddings in various ways, and query the quantum solver to a different extent, a breakdown of computational time across the three categories is useful for detecting computational bottlenecks and general trends for different QUBO problem graphs.

3. *Solution quality*. All QUBO problem terms are generated from problem constraints and an objective function. For each problem described in Section 2.2, *solution energy* refers to the energy of a given solution, which is optimal when minimized. The number of *broken constraints* refers to how many problem-specific constraints a given solution violates. For example, a TSP solution that does not visit all nodes exactly once violates one or more constraints that define a feasible solution.

   For each problem, a method for recording and reconciling broken constraints, thereby "fixing" an infeasible solution (via greedy post-processing), was implemented. We define the quality of a "fixed" solution, referred to as *fixed solution quality*, uniquely for each problem type, since a fixed solution is feasible and can therefore be assigned a problem-relevant measure of quality. For the DBG, TSP, SCA, and MWP problems, the fixed solution quality is, respectively, a generalized reward value, the path length, the total sub-constellation coverage, and a measure of the value of completed jobs. Note that TSP fixed solution quality is better when minimized (i.e., shortest route is ideal), while fixed solution quality for DBG, SCA, and MWP is better when maximized.

3.3 QHOT parameter testing

Since the ultimate goal of this research was to evaluate and compare the performance of various QHOTs, it was imperative that we understood the performance of the QHOTs within their own parameter spaces. By conducting this analysis, it allowed us to select the "best" combination of parameter values for a given QHOT to use as a benchmark when comparing against the other techniques. In this work, the two solvers with variable parameters (FA and Qbsolv) were tested.

Figure 2 highlights a clear relationship between the total runtime of the FA algorithm and the product of the two parameters: number_of_generations and population_size. This increase in runtime is expected and results directly from an increase in classical computational time; increasing the number_of_generations increases the overall number of classical iterations of the GA, while increasing population_size similarly requires the evaluation of a larger number of solutions per iteration. Analyzing the fixed solution quality results, there is no clear strong performance advantage in choosing parameters that increase FA runtimes in our experiments on DBG problems. This is especially evident by the near-parallelism of the lines of best fit for each combination of parameter values. Therefore, for the FA experiments, we chose to use the parameter values that resulted in the fastest runtimes—that is, population_size set to 250 and number_of_generations set to 10.

A similar trend was observed in Fig. 3 in terms of the relationship between runtime and fixed solution quality. As NumRepeats increases, time-to-solution increases, generally. For each value of NumRepeats tested, the rate at which longer runtime increased fixed solution quality (indicated by the slopes of the lines of best fit) also increased, marginally. However, this increase in slope is a result of selection bias caused by our time threshold: fewer experiments were able to complete in 30 min as NumRepeats increased. For example, setting NumRepeats to 50 (the current default for this parameter) greatly reduced the number of DBG and TSP problems solvable and made the MWP and SCA problems unsolvable within the given time threshold. With that, and the marginal improvements seen from increasing NumRepeats, we set NumRepeats to 1 for all Qbsolv experimental comparisons to maximize the algorithm's ability to return a solution in a timely manner.

3.4 Performance of QHOTs

Using the FA and Qbsolv parameter values determined in Section 3.3, we evaluated the timing and performance across each of our QHOTs. Figure 4 breaks down timing information, per solver, for all DBG and TSP problems solvable within the 30-min upper-bound time threshold.

Interesting patterns emerge in comparing the timing breakdowns of the PCD, FA, Qbsolv, and ICH solvers, which were predominately dominated by embedding time, classical time, quantum time, and a combination of both quantum and embedding times, respectively. These results are best explained as a direct result of algorithm design:

1. The majority of FA's runtime is spent classically optimizing via the GA before finally submitting a single reduced QUBO to the quantum annealer.
2. PCD was designed to attempt to find an embedding of a sub-problem, further slicing this sub-problem along the primary axis if an embedding was not found. The algorithm used to find graph embeddings does not scale well with graph size and lacks adequate early dropout mechanisms—hence, the majority of PCD's

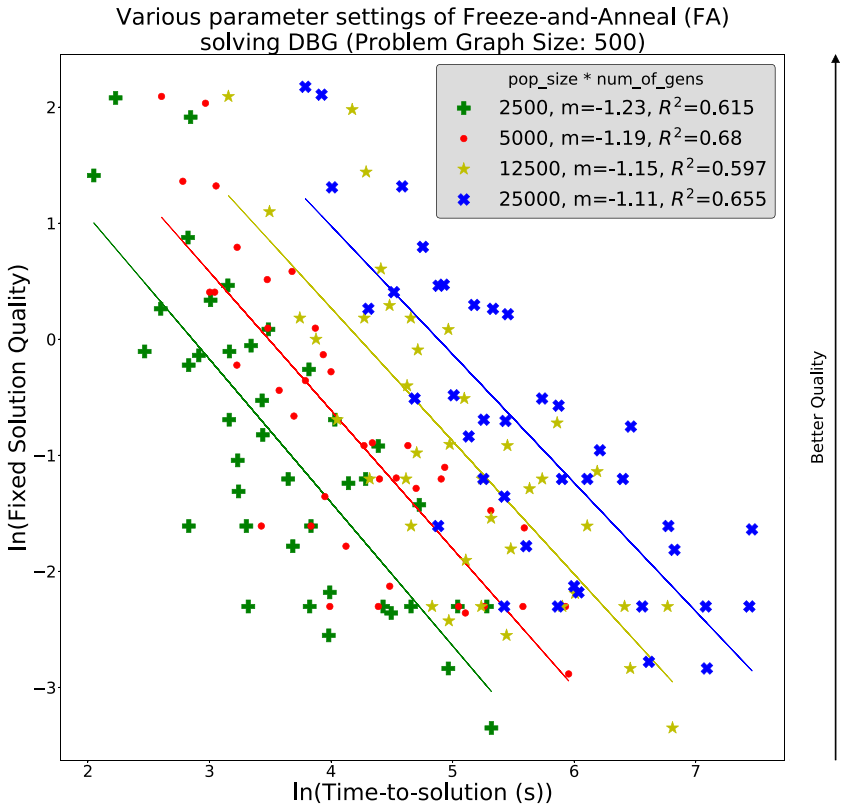


Fig. 2 Comparing time-to-solution and fixed solution quality for various parameter settings of the FA algorithm solving DBG problems. Similar results were observed for DBG problems of size 1000. This figure highlights a clear relationship between the total runtime of the FA algorithm and the product of the two parameters: number_of_generations and population_size. This increase in runtime is expected and results directly from an increase in classical computational time. Analyzing the fixed solution quality results, there is no clear strong performance advantage in choosing parameters that increase FA runtimes in our experiments on DBG problems. This is especially evident by the near-parallelism of the lines of best fit for each combination of parameter values. Therefore, for the FA experiments, we chose to use the parameter values that resulted in the fastest runtimes—that is, population_size set to 250 and number_of_generations set to 10

time is spent searching for, and often failing to find, embeddings.

3. Due to Qbsolv's black-box implementation (being an externally developed algorithm), it was difficult to classify how the algorithm's time was being computationally spent. Wrapping function calls made by the Qbsolv algorithm allowed for accurate recording of quantum time. Unfortunately, we could not divide the remainder of Qbsolv's time into classical and embedding, so we report this non-quantum time as a combination of the two. The majority of Qbsolv's relative time is consumed by calls to the quantum device, with non-quantum time becoming noticeable for problems with larger size and/or higher edge density. It is likely that Qbsolv's non-quantum time is almost entirely classical, since the algorithm is designed to reuse a single problem embedding.
4. Due to ICH's relatively straightforward classical component, the majority of its time is spent embedding and solving sub-problems. Despite not reusing a single problem embedding (as Qbsolv does), the relative embedding time of ICH is considerably less than that of PCD. This is because ICH is designed such that an embedding is guaranteed to exist for any sub-problem to be solved; therefore, no time is wasted waiting for the heuristic embedding finder to fail.

As expected for larger problem graphs with greater size and edge density (e.g., larger TSP problems), all QHOTs eventually struggled to find solutions within the time threshold. In terms of overall experiment completion, the FA, Qbsolv, and ICH algorithms were able to solve almost all of the same problems within the time threshold.

In terms of time-to-solution for DBG and TSP, the results shown in Fig. 5 echo those of Fig. 4. As expected, all QHOTs require more time as problem size increases. In particular, PCD scales poorly as problem size and edge density increase. The same is true, to a lesser degree, for FA, which requires more time as edge density increases because graphs with higher edge density require more classical

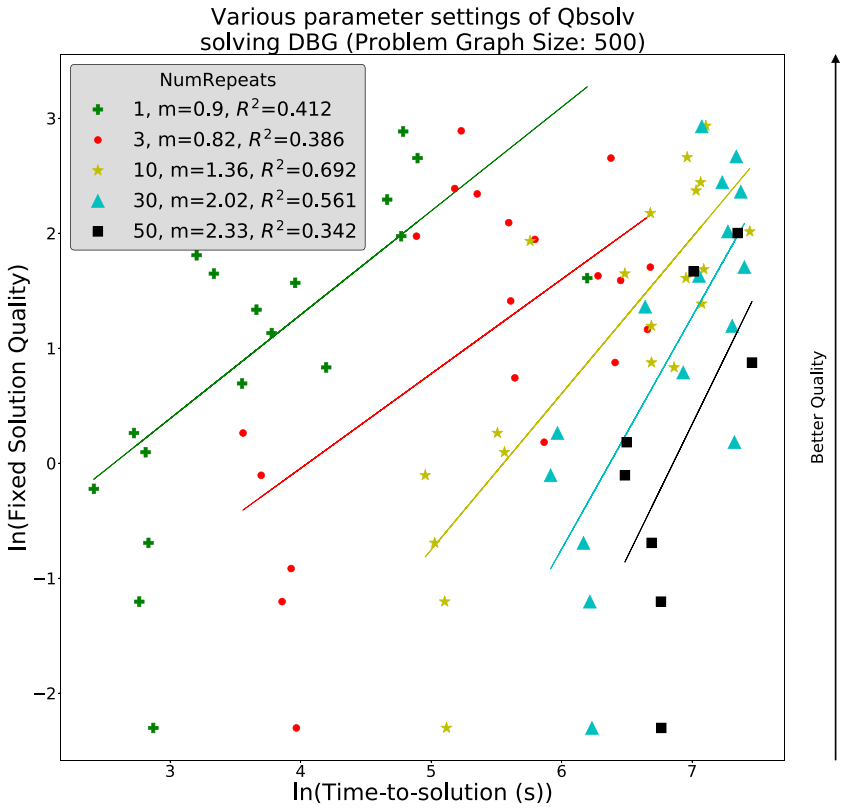


Fig. 3 Comparing time-to-solution and fixed solution quality for various parameter settings of the Qbsolv algorithm solving DBG problems. Similar results were observed for DBG problems of size 1000. As NumRepeats increases, time-to-solution increases, generally. For each value of NumRepeats tested, the rate at which longer runtime increased fixed solution quality (indicated by the slopes of the lines of best fit) also increased, marginally. However, this increase in slope is a result of selection bias caused by our time threshold: fewer experiments were able to complete in 30 min as NumRepeats increased. For example, setting NumRepeats to 50 (the current default for this parameter) greatly reduced the number of DBG and TSP problems solvable and made the MWP and SCA problems unsolvable within the given time threshold. With that, and the marginal improvements seen from increasing NumRepeats, we set NumRepeats to 1 for all Qbsolv experimental comparisons to maximize the algorithm's ability to return a solution in a timely manner

processing to propagate information of frozen qubits to adjacent variables. Conversely, ICH requires less time as edge density increases. Given a central node (as defined in the ICH algorithm), a higher overall problem graph edge density implies that that central node has closer to max_nodes neighbors. Therefore, while sub-problems are larger, on average, the total number of sub-problems to be solved is fewer, thus resulting in a decrease in overall solving time. Lastly, Qbsolv's time-to-solution, as opposed to that of the other QHOTs, is the least correlated with edge density and graph size. This is likely due to the fact that randomness is fundamental to the Qbsolv algorithm. While the same can be said about FA with its incorporation of a GA, the quality of randomly generated solutions has a direct effect on Qbsolv's termination and, therefore, its runtime.

In terms of the quality of solutions to the DBG and TSP problems shown in Fig. 6, Qbsolv produced the best solutions for every problem that it was able to solve within the time limit. This is likely because the number of broken constraints a solution has is directly correlated with solution quality, and the solutions that Qbsolv found, while not necessarily optimal, never had broken constraints. This ability to always return viable solutions to TSP problems can be seen in the righthand plot of Fig. 6, though it is also the case for all DBG problems tested. Solutions found using the ICH algorithm typically had the second-best quality, behind Qbsolv. PCD outperformed both the FA and RGCP solvers, but only produced solutions for problems with limited edge density and size. Lastly, of the QHOTs evaluated in this work, FA produced the lowest quality solutions to the DBG and TSP problems, with quality comparable to, though slightly higher than, that of solutions returned by RGCP.

Also evident from our results is the difficulty, across all solvers, in finding solutions to TSPs with 10,000+ qubits (i.e., 100+ nodes to visit) in a timely fashion. Of the 6 100-node TSPs (1 Random, 5 from TSPLIB), 1 was solved in 30 min, and by only a single QHOT.

QHOT performance results when solving the MWP and SCA problems are shown in Table 1. Similar to both DBG

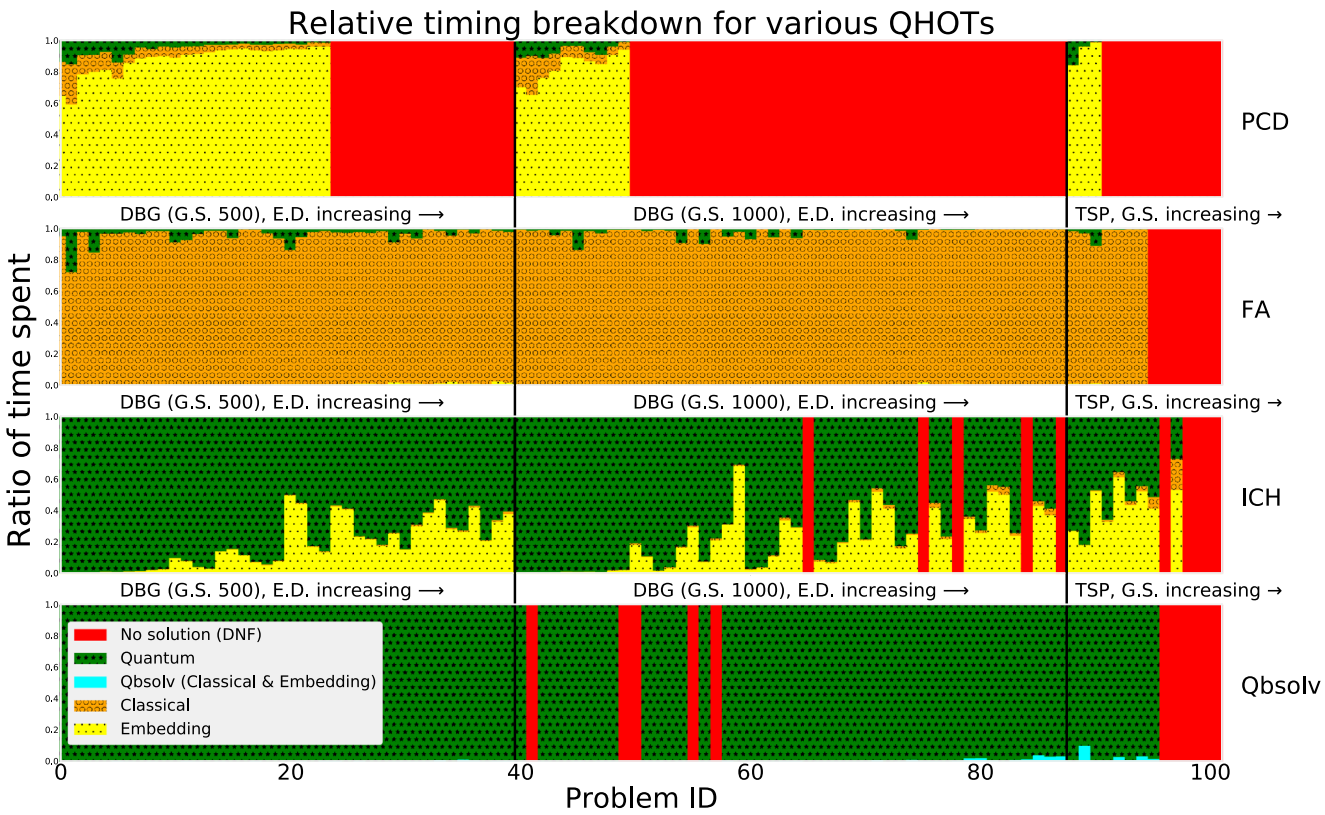


Fig. 4 Relative timing breakdown for various QHOTs on DBG and TSP problems. Bars represent unique problems being solved and are arranged along the x-axis in the following order: DBG problems with graph size (G.S.) = 500 and edge density (E.D.) increasing, DBG problems with G.S. = 1000 and E.D. increasing, and TSP problems with G.S. increasing. A black bar is used to visually separate these sets of problems

and TSP problems, Qbsolv outperforms all other QHOTs in terms of both time-to-solution and solution energy, for the MWP and SCA problems. ICH places second for each of the aforementioned categories, except MWP time-to-solution in which it places last. This is likely due to the MWP having a very sparse problem graph, which ICH struggles with.

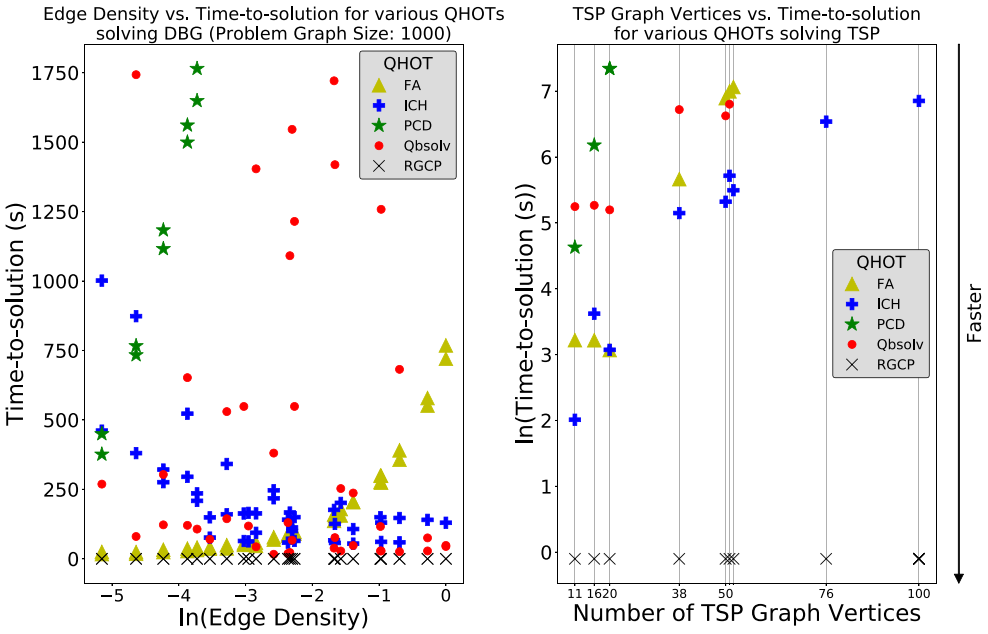


Fig. 5 Comparing time-to-solution against both edge density and problem graph size for various QHOTs solving DBG (1000-node) and TSP problems. Similar results were observed for DBG problems of size 500

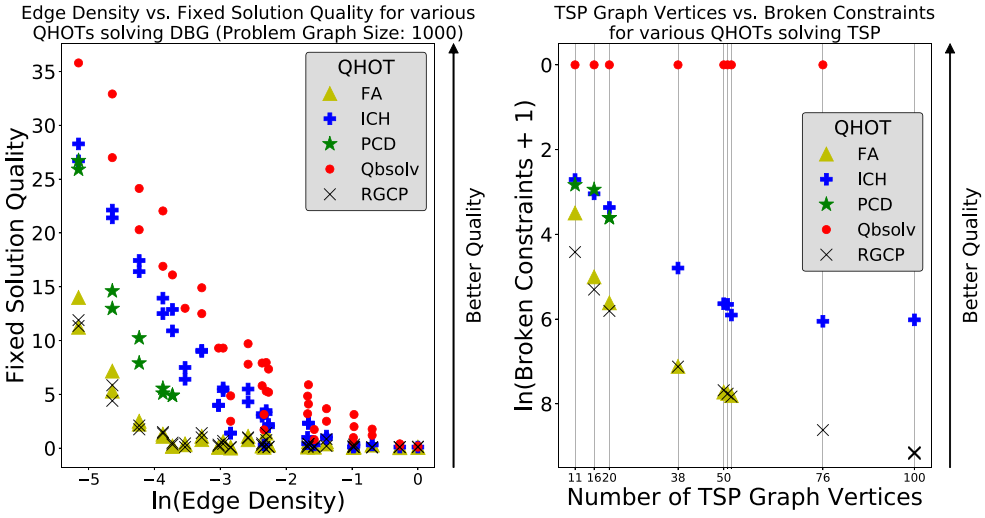


Fig. 6 Comparing solution quality against both edge density and problem graph size for various QHOTs solving DBG (1000-node) and TSP problems. Similar results were observed for DBG problems of size 500

4 Discussion

Quantum computers, and quantum annealers in particular, offer the possibility of providing performance improvements in the domain of numerical optimization. This domain has many important real-world applications, such as the satellite coverage optimization and repair job scheduling problems discussed in this paper. While it is considered unlikely that quantum computers will provide an exponential improvement in this area, polynomial improvements are possible, and even constant pre-factor improvements could be of great societal impact given the huge benefit of even slight improvements for some of these problems.

In order to explore these possibilities, it is essential to begin exploring real-world optimization problems on physical devices as soon as possible. However, current commercially available quantum annealing devices are highly limited in terms of numbers of qubits and connectivity, and are unable to be directly tested on real-world problems of relevant size. As such, decomposition techniques are needed.

The QHOTs presented in this work are the first step to understanding how best to use existing quantum annealing devices to solve real-world problems. They displayed the capability of finding solutions to problems consisting of hundreds and even thousands of problem variables, which are considerably larger than the problem graphs directly embeddable onto current generation quantum devices. Even larger problems may be solved and solution quality increased by analyzing some key takeaways from this work and extending current and future QHOTs to maximize their utility.

The approaches laid out here are far from fully optimized against technologies such as CPLEX and other state-of-the-art optimization solvers. To approach relevancy against state-of-the-art solvers, heterogenous algorithms will need to simultaneously improve quantum hardware and reduce resource requirements using classical techniques. Improvements in quantum hardware will result in larger hardware graphs with higher connectivity that would significantly increase the size of decomposed subproblems for

Table 1 Experimental results for all QHOTs on MWP and SCA problems

Problem (size, edge density)	Solver	Total time (s)	Solution energy	Broken constraints
SCA (1013, 1)	ICH	260.295	56,794,709	1674
	FA	249.714	458,850,157	75,413
	PCD	1105.106	962,418,755	331,662
	Qbsolv	139.148*	1,949,602*	0*
	RGCP	0.079	470,288,031	79,067
MWP (2280, 0.0026)	ICH	10,538	-1.356×10^5	N/A
	FA	1350.49	−67,160.10	N/A
	PCD	849.87	-1.313×10^5	N/A
	Qbsolv	40.03*	-1.357×10^{5}*	N/A
	RGCP	0.0029	−53,025.93	N/A

*Indicates best performance

solving directly on the hardware. In terms of resource reductions, classical techniques will be useful for limiting run times (e.g., reuse of QUBO embeddings) as well as memory requirements (converting optimization problems into QUBO format can scale memory requirements quadratically). More exotic algorithmic improvements, such as parallelizing heterogenous algorithms to solve several subproblems on different hardware devices, could also help close the gap between such techniques and industry-standard solvers.

Sub-QUBO information propagation. For the solvers that produced the highest quality solutions—namely, ICH and Qbsolv—sub-QUBO solutions are iteratively used to update the remaining global QUBO problem while determining the global solution. This "information propagation" is not harnessed, in any capacity, by the PCD and Random solvers, which produced solutions of worse quality than ICH and Qbsolv on all problems evaluated in this work. A modified version of this information propagation is implemented by the FA algorithm, which produced solutions with qualities similar to the Random solver. However, while Qbsolv and ICH propagated information gained from sending sub-QUBOs to the quantum annealer, FA propagated only classically determined information gained after the GA had optimized over the entire solution space.

Reuse of static embeddings. As evidenced by the PCD and ICH results of Fig. 4, constantly recalculating hardware embeddings become a major computational bottleneck as problem size and edge density increase. Significant temporal resources are consumed when heuristically searching for an embedding. This is especially true for sub-problems that are impossible to embed due to their size or edge density because the algorithm that minorminer implements scales poorly as problem graph size increases and often consumes a significant amount of time before returning an indication of failure in the event that an embedding cannot be found.

To avoid this issue, one pragmatic approach for use in future QHOTs is to pre-calculate the hardware embedding for the largest fully connected problem graph that can be embedded onto an available quantum device (this is 65 for the 2000Q). Rather than repeatedly spending time searching for embeddings, this single embedding can be reused by trivially mapping any sub-problem with equal or lesser size to it. Qbsolv takes an approach similar to this, which explains the minimal non-quantum time usage. One drawback of this approach is that sub-problems of a greater size with less-than-fully-connected problem graphs may be embeddable, but are further reduced.

However, alleviating this drawback introduces the risk of wasting time searching for an embedding that does not exist which, given current embedding finder heuristics, greatly reduces the efficiency of QHOTs that accept this risk (e.g., PCD). Additionally, general improvements in the heuristic embedding search algorithm (increased speed, more advanced dropout for problems that cannot be embedded, etc.) will naturally result in improvements in time-to-solution for QHOTs that require recalculation of hardware embeddings.

Algorithmic improvements can be had by introducing these techniques (sub-QUBO information propagation and embedding reuse) to those QHOTs that do not leverage both. Additionally, as mentioned in the "Introduction," improvements may arise from employing different classical optimization algorithms (e.g., Monte Carlo methods) other than the GA (used by FA) and the Tabu search (used by Qbsolv). A more rigorous examination of the full range of classical optimization techniques was beyond the scope of this work, but further work investigating this area could prove fruitful.

It is also likely that dynamically repairing broken constraints using either problem-specific or problem-agnostic information during algorithm execution, rather than after, will produce higher quality solutions, and is therefore a research area that should be explored. Beyond improving the QHOTs that we explored, there are certainly other decomposition techniques that should be evaluated. For example, the techniques used in Ajagekar et al. (2020) can be ported into our framework and generalized to solve arbitrary QUBO problems.

Lastly, it has been shown that, when solution quality is solely considered, Qbsolv (with NumRepeats set to 1) is superior for solving both theoretical (DBG, TSP) and real-world (MWP, SCA) problems. However, Qbsolv's time-to-solution relies heavily on randomness; and therefore, Qbsolv is the most temporally unpredictable of the QHOTs explored in this work. Currently, the default value for Qbsolv's NumRepeats parameter is 50. Given the marginal improvements in solution quality that increasing NumRepeats has shown, it is likely that one can comfortably, in the interest of time, afford to decrease NumRepeats to a value dependent on the criticality of time-to-solution with respect to solution quality.

Acknowledgements The authors would like to acknowledge the support of the Universities Space Research Association, Quantum AI Lab Research Opportunity Program, Cycle 2, for providing us with remote access to a D-Wave 2000Q device. Additionally, the thoughtful reviews and suggestions from Duncan Fletcher are acknowledged.

We would also like to thank the anonymous reviewers for the helpful comments and suggestions which have significantly improved the manuscript.

Funding This work was funded through the Air Force Research Laboratory, Contract No. FA8750-18-C-0167.

References

Ajagekar A, Humble T, You F (2020) Quantum computing based hybrid solution strategies for large-scale discrete-continuous optimization problems. Comput Chem Eng 132(607):1–50. ISSN 00981354. https://doi.org/10.1016/j.compchemeng.2019.106630

Arute F et al (2019) Quantum supremacy using a programmable superconducting processor. Nature 574(7779):505–510. ISSN 14764687. https://doi.org/10.1038/s41586-019-1666-5

Bass G, Tomlin C, Kumar V, Rihaczek P, Dulny J (2018) Heterogeneous quantum computing for satellite constellation optimization: solving the weighted k-clique problem. Quantum Sci Technol 3(2):24010. ISSN 2058-9565. https://doi.org/10.1088/2058-9565/aaadc2

Bian Z, Chudak F, Israel R, Lackey B, Macready WG, Roy A (2014) Discrete optimization using quantum annealing on sparse Ising models. Front Phys 2:56. ISSN 2296-424X. https://doi.org/10.3389/fphy.2014.00056

Bian Z, Chudak F, Israel R, Lackey B, Macready WG, Roy A (2016) Mapping constrained optimization problems to quantum annealing with application to fault diagnosis. http://arxiv.org/abs/1603.03111

Booth M, Reinhardt SP, Roy A (2017) Partitioning optimization problems for hybrid classical/quantum execution. www.dwavesys.com

Brooke J, Bitko D, Rosenbaum T˜F, Aeppli G (1999) Quantum annealing of a disordered magnet. Science 284:779. https://doi.org/10.1126/science.284.5415.779

Cai J, Macready W˜G, Roy A (2014) A practical heuristic for finding graph minors. ArXiv e-prints

Chapuis G, Djidjev H, Hahn G, Rizk G (2019) Finding maximum cliques on the D-wave quantum annealer. J Signal Process Syst 91(3-4):363–377. ISSN 1939-8018. https://doi.org/10.1007/s11265-018-1357-8

D-Wave (2018) The D-Wave 2000Q System

Djidjev HN, Chapuis G, Hahn G, Rizk G (2018) Efficient combinatorial optimization using quantum annealing. pp 1–25, http://arxiv.org/abs/1801.08653

Djidjev HN, Hahn G, Mniszewski SM, Negre CFA, Niklasson AMN, Sardeshmukh VB (2016) Graph partitioning methods for fast parallel quantum molecular dynamics. http://arxiv.org/abs/1605.01118

Farhi E, Goldstone J, Gutmann S, Lapan J, Lundgren A, Preda D (2001) A quantum adiabatic evolution algorithm applied to random instances of an NP-Complete problem. Science 292(5516):472. http://science.sciencemag.org/content/292/5516/472.abstract

Finnila A˜B, Gomez M˜A, Sebenik C, Stenson C, Doll J˜D (1994) Quantum annealing: a new method for minimizing multidimensional functions, vol 219

Fruchterman TMJ, Reingold EM (1991) Graph drawing by force-directed placement. Software: Practice and Experience 21(11):1129–1164. ISSN 0038-0644. https://doi.org/10.1002/spe.4380211102

Garey MR, Johnson DS (1990) Computers and intractability; a guide to the theory of NP-completeness. W. H. Freeman & Co., New York. ISBN 0716710455

Goldberg DE, Holland JH (1988) Genetic algorithms and machine learning. Mach Learn 3(2):95–99. ISSN 1573-0565. https://doi.org/10.1023/A:1022602019183

Hagberg AA, Schult DAS, Swart PJ (2008) Exploring network structure, dynamics, and function using NetworkX. In: Proceedings of the 7th python in science conference (SciPy2008)

Henderson M, Novak J, Cook T (2019) Leveraging quantum annealing for election forecasting. J Phys Soc Japan 88(6):061009. https://doi.org/10.7566/JPSJ.88.061009

Holland JH (1992) Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence, 2nd edn. MIT Press, Cambridge

Job Joshua, Adachi Steve (2020)

Kadowaki T, Nishimori H (1998) Quantum annealing in the transverse Ising model. Phys Rev E 58:5355–5363. https://doi.org/10.1103/PhysRevE.58.5355

Lucas A (2014) Ising formulations of many NP problems. Front Phys 2:5. https://doi.org/10.3389/fphy.2014.00005

Neven H, Denchev VS, Rose G, Macready WG (2012) Qboost: Large scale classifier training with adiabatic quantum optimization. In: Hoi SCH, Buntine W (eds) Proceedings of the Asian conference on machine learning, vol 25 of proceedings of machine learning research. Singapore Management University, Singapore, pp 333–348. PMLR. http://proceedings.mlr.press/v25/neven12.html

Noori M, Vedaie SS, Singh I, Crawford D, Oberoi JS, Sanders BC, Zahedinejad E (2020) Analog-quantum feature mapping for machine-learning applications. Phys Rev Appl 14:034034. https://doi.org/10.1103/PhysRevApplied.14.034034

Preskill J (2018) Quantum Computing in the NISQ era and beyond. https://doi.org/10.22331/q-2018-08-06-79. arXiv:1801.00862

Reinelt G (1995) Tsplib 95. interdisziplinäres Zentrum für Wissenschaftliches Rechnen (IWR). Heidelberg 338:1–16

Salakhutdinov R, Hinton G (2009) Deep Boltzmann machines. Aistats 1(3):448–455. ISSN 1063-6919. https://doi.org/10.1109/CVPRW.2009.5206577

Santoro G˜E, Marto\v nák R, Tosatti E, Car R (2002) Theory of quantum annealing of an Ising spin glass. Science 295:2427–2430. https://doi.org/10.1126/science.1068774

Santoro GE, Tosatti E (2006) Optimization using quantum mechanics: quantum annealing through adiabatic evolution. J Phys A: Math Gen 39(36):R393. http://stacks.iop.org/0305-4470/39/i=36/a=R01

Squires RR, Hoffman KL (2014) A military maintenance planning and scheduling problem. Optim Lett 9(8):1675–1688. ISSN 18624480. https://doi.org/10.1007/s11590-014-0814-y

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.